

Jenkins Distributed Architecture

Complete End-to-End Guide

Master · Agents · Pipelines · Kubernetes · AWS

Terraform · Docker · Amazon EKS · GitHub Webhooks · CI/CD

DevOps Engineering Reference | 2024

Table of Contents

What's inside this guide

1	Architecture Overview	The 1-master + N-agents model
2	Jenkins Agents Deep Dive	What each agent does & why
3	Pipeline Stages	Frontend, Backend & Terraform
4	Jenkinsfile Reference	Complete annotated pipeline code
5	Deployment Flow	Branch strategy & environment routing
6	Kubernetes & EKS	Namespaces, clusters & deployments
7	GitHub Integration	Webhooks, branches & secrets
8	Real-World Scale	Managing 20–100+ pipelines
9	Interview Answers	Prepared answers for common questions

1. Architecture Overview

How GitHub, Master, and Agents connect

Your senior is describing a **distributed Jenkins architecture** — one Jenkins Master (Controller) that orchestrates everything, and dedicated Agent servers that execute the actual work. This pattern is standard in production DevOps because it delivers separation of concerns, security isolation, and horizontal scalability.

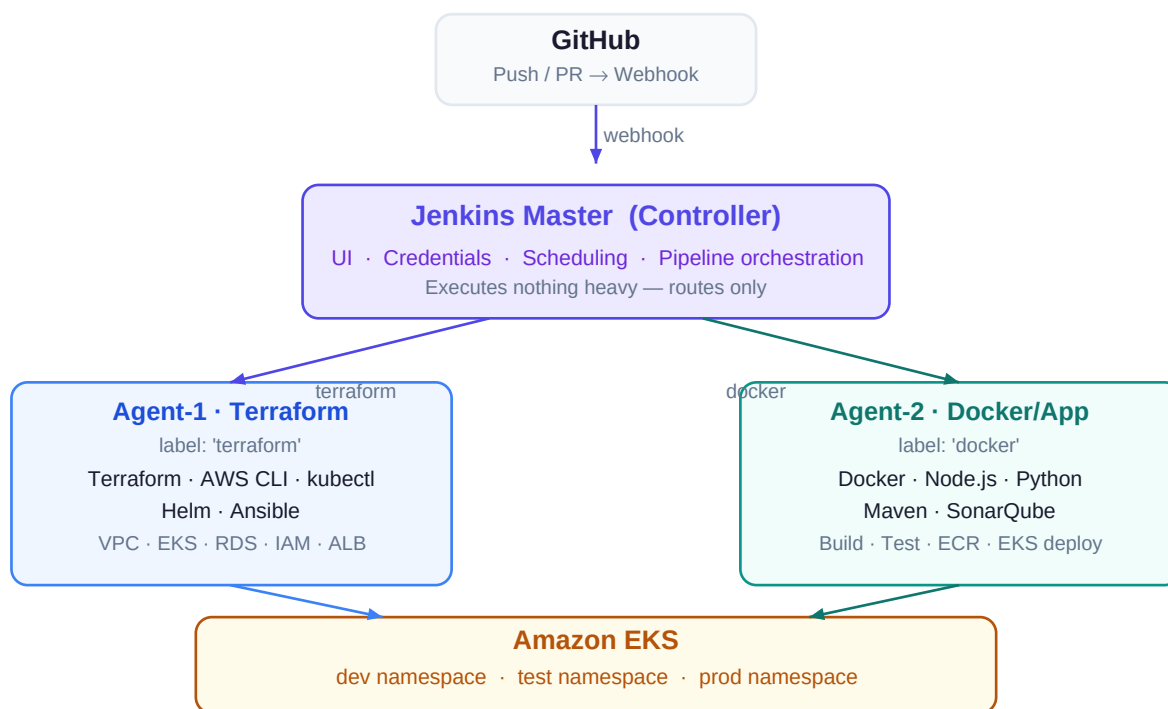


Figure 1 — Jenkins distributed architecture: GitHub webhook → Master → Agents → EKS

Core components

Jenkins Master	Orchestrates pipelines, stores credentials, hosts the UI. Executes zero heavy jobs.
Agent-1 · Terraform	Dedicated infrastructure agent. Runs Terraform, AWS CLI, kubectl, Helm.
Agent-2 · App/Docker	Dedicated build agent. Runs Docker, Node.js, Python, Maven, SonarQube.
Amazon EKS	Target Kubernetes cluster with dev, test, and prod namespaces.

2. Jenkins Agents Deep Dive

Labels, connection methods & tool isolation

How agents connect to the master

Each agent is registered in Jenkins with a **label**. The Jenkinsfile declares agent `{ label 'terraform' }` and the master finds any free agent with that label and routes the stage to it. Communication happens via:

- **SSH** — most common for Linux agents (port 22)
- **JNLP / WebSocket** — inbound agent, agent initiates connection
- **Kubernetes pods** — dynamic cloud agents (advanced)

Server	Role	Label	Tools Installed	Responsibilities
Jenkins Master	Controller	—	Jenkins, JDK, Git	UI, credentials, scheduling, webhook receiver, pipeline store
Agent-1	Infrastructure	terraform	Terraform, AWS CLI, kubectl, Helm, ArgoCD	terraform init/plan/apply, VPC, EKS, RDS, IAM, ALB
Agent-2	Application	docker	Docker, Node.js, Python, Maven, Gradle, SonarQube	Docker build, push ECR, kubectl deploy

Why separate agents?

Performance — Heavy builds don't overload the Jenkins controller.

Security — AWS credentials are isolated to Agent-1 — Agent-2 never sees infra keys.

Tool isolation — Terraform/Docker/Python environments don't conflict.

Scalability — Multiple jobs run in parallel on different agents simultaneously.

Maintainability — Each agent is purpose-built and independently upgradeable.

3. Pipeline Stages

Frontend, Backend & Terraform — side by side

Each repository has its own Jenkinsfile and therefore its own pipeline. Three core pipelines drive the full application lifecycle:

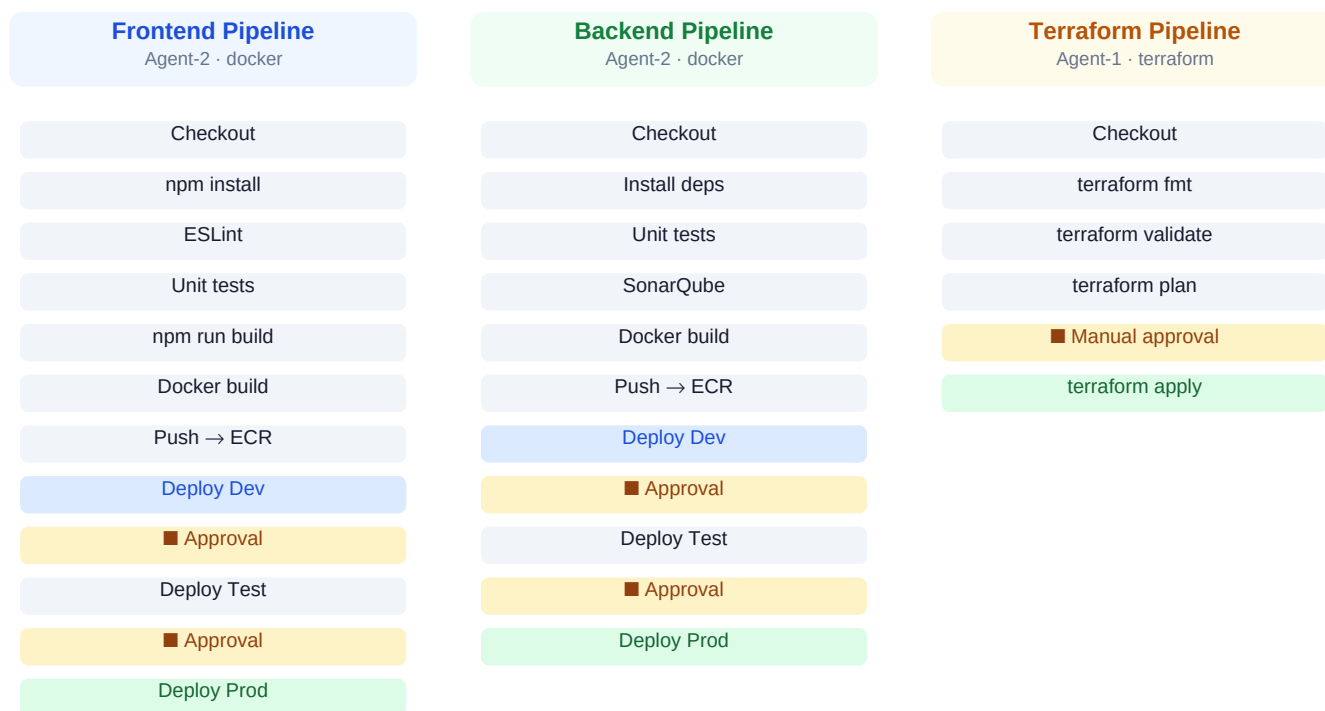


Figure 2 — Three pipelines running on dedicated agents with approval gates before Test and Production deploys

Key pipeline concepts

Concept	Explanation
agent none at root	No default agent — each stage picks its own. The master executes nothing.
agent { label '...' }	Routes a stage to a specific agent by label. Master finds a free matching agent.
when { branch '...' }	Stage only runs when the branch name matches. Prevents dev code deploying to prod.
input 'Approve?'	Pauses the pipeline and waits for a human to click Approve before proceeding.
SonarQube stage	Static code analysis on Agent-2. Fails the pipeline if quality gate is not met.

4. Jenkinsfile Reference

Complete annotated pipeline code

```

pipeline {

    agent none // Master executes nothing; stages pick their own agent

    stages {

        // ■■ Terraform stages → Agent-1 (label: terraform) ■■■■■■■■■■
        stage('Terraform Init') {
            agent { label 'terraform' }
            steps { sh 'terraform init' }
        }
        stage('Terraform Plan') {
            agent { label 'terraform' }
            steps { sh 'terraform plan' }
        }
        stage('Terraform Apply') {
            agent { label 'terraform' }
            steps { sh 'terraform apply -auto-approve' }
        }
    }

    // ■■ App build stages → Agent-2 (label: docker) ■■■■■■■■■■
    stage('Unit Tests') {
        agent { label 'docker' }
        steps { sh 'npm test' } // or pytest, mvn test
    }
    stage('Docker Build & Push') {
        agent { label 'docker' }
        steps {
            sh 'docker build -t myapp:${BUILD_NUMBER} .'
            sh 'docker push 123456789.dkr.ecr.ap-south-1.amazonaws.com/myapp:${BUILD_NUMBER}'
        }
    }

    // ■■ Deploy Dev (no approval) ■■■■■■■■■■
    stage('Deploy Dev') {
        when { branch 'develop' }
        agent { label 'docker' }
        steps {
            sh 'kubectl config use-context dev'
            sh 'kubectl apply -f k8s/'
        }
    }

    // ■■ Deploy Test (approval required) ■■■■■■■■■■
    stage('Deploy Test') {
        when { branch 'release' }
        agent { label 'docker' }
        steps {
            input 'Approve deploy to Test?'
            sh 'kubectl config use-context test'
        }
    }
}

```

```
        sh 'kubectl apply -f k8s/'
    }
}

// ■■ Deploy Production (manual approval + reviewers) ■■■■■■■■■■
stage('Deploy Production') {
    when { branch 'main' }
    agent { label 'docker' }
    steps {
        input 'Approve PRODUCTION deployment?'
        sh 'kubectl config use-context prod'
        sh 'kubectl apply -f k8s/'
    }
}
}
```

Listing 1 — Complete Jenkinsfile for a full multi-environment deployment

5. Deployment Flow

Branch strategy and environment routing

Branch strategy

Branch	Environment	Trigger	Approval required?
feature/*	—	PR only	None
develop	Dev namespace	Git push → webhook	None — auto deploy
release	Test namespace	Merge develop → release	Required before deploy
main	Production	Merge release → main	Manual + required reviewers

End-to-end journey of a single code change

Step	Action	Detail
1	Developer pushes to develop	git push origin develop
2	GitHub fires webhook	HTTP POST to Jenkins master endpoint
3	Master evaluates Jenkinsfile	Reads branch, decides which stages apply
4	Terraform pipeline (Agent-1)	terraform init → plan → apply → VPC/EKS/RDS/ALB created
5	App pipeline (Agent-2)	npm/pytest → Docker build → push image to Amazon ECR
6	Deploy to dev namespace	kubectl apply -f k8s/ — no approval needed
7	Merge develop → release	Triggers test pipeline, approval gate pauses
8	Approval given	Pipeline resumes, deploys to test namespace
9	Merge release → main	Approval required → deploy to production namespace

6. Kubernetes & Amazon EKS

Cluster structure, namespaces & deployments

Two common patterns for multi-environment Kubernetes:

Pattern	Structure	When to use
Single cluster, multiple namespaces	One EKS cluster with dev / test / prod namespaces	Cost effective for smaller teams. Namespaces provide logical isolation.
Multiple clusters	EKS Dev Cluster + EKS Test Cluster + EKS Prod Cluster	Stark isolation required — prod failures cannot impact dev. Enterprise standard.

Kubernetes manifests repository structure

```
k8s/
├── frontend/
│   ├── deployment.yaml # image: ECR_REPO/frontend:${BUILD_NUMBER}
│   ├── service.yaml    # ClusterIP / LoadBalancer
│   └── ingress.yaml     # ALB Ingress Controller
├── backend/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── configmap.yaml
└── namespace.yaml      # dev / test / prod
```

Terraform provisions:

- VPC with public and private subnets
- Internet Gateway and NAT Gateway
- Security Groups (ALB, EKS nodes, RDS)
- IAM Roles for EKS worker nodes and service accounts
- Application Load Balancer
- Amazon EKS Cluster + node groups
- Amazon RDS (PostgreSQL / MySQL)
- Amazon ECR repositories for Docker images

7. GitHub Integration

Webhooks, repo structure & environment secrets

Repository structure

```
GitHub Organization
├── frontend-repo/
│   ├── src/
│   ├── Dockerfile
│   ├── Jenkinsfile      ← frontend pipeline
│   └── k8s/
├── backend-repo/
│   ├── app.py
│   ├── requirements.txt
│   ├── Dockerfile
│   ├── Jenkinsfile      ← backend pipeline
│   └── k8s/
├── terraform-repo/
│   ├── modules/
│   ├── env/
│   ├── dev.tfvars
│   ├── test.tfvars
│   ├── prod.tfvars
│   └── Jenkinsfile      ← terraform pipeline
```

GitHub environment secrets

Environment	Secrets stored
Development	AWS_ACCESS_KEY, AWS_SECRET_KEY, AWS_REGION, KUBECONFIG_DEV, DATABASE_URL
Testing	AWS_ACCESS_KEY, AWS_SECRET_KEY, AWS_REGION, KUBECONFIG_TEST
Production	AWS_ACCESS_KEY, AWS_SECRET_KEY, AWS_REGION, KUBECONFIG_PROD + protection rules

Production environment protection rules include: **Manual Approval** required and **Required Reviewers** — the pipeline pauses until a named reviewer approves via GitHub.

8. Real-World Scale

Managing 20–100+ pipelines in production

Example: E-commerce platform with 8 pipelines

Pipeline	Agent	Purpose
pipeline-frontend	Agent-2	Build and deploy React application
pipeline-backend	Agent-2	Build and deploy REST API
pipeline-payment	Agent-2	Deploy payment microservice
pipeline-auth	Agent-2	Deploy authentication service
pipeline-infra	Agent-1	Run Terraform — provision AWS
pipeline-monitoring	Agent-2	Deploy Prometheus & Grafana
pipeline-db	Agent-2	Database schema migrations
pipeline-security	Agent-2	Trivy scan, SAST, dependency audit

Jenkins folder organization

Jenkins organizes pipelines into **Folders**. Two common patterns:

By Environment	By Team
■ Development ■ frontend · backend · terraform ■ QA ■ frontend · backend · terraform ■ Production ■ frontend · backend · terraform	■ DevOps ■ terraform · monitoring ■ Backend Team ■ api · auth · payment ■ Frontend Team ■ react · admin

Large enterprise agent pool

Large organizations typically run multiple agents per type to allow parallel jobs:

Agent Type	Count	Purpose
Linux build agents	10	Docker builds, npm, pytest, Maven
Terraform agents	4	Infrastructure provisioning only
Kubernetes agents	6	kubect!, Helm deployments
Security scan agents	2	Trivy, Snyk, SAST tools
Windows agents	3	Windows builds, .NET, PowerShell

9. Interview Answers

Word-for-word answers to common DevOps questions

Q: Explain your Jenkins architecture.

We use a distributed Jenkins setup — one Jenkins Master that handles orchestration, credentials, and the UI, plus dedicated agents for different workloads. Agent-1 runs Terraform and handles all AWS infrastructure provisioning. Agent-2 handles builds, testing, Dockerizing, and Kubernetes deployments. The master itself executes nothing heavy — it just schedules and routes stages to the right agent by matching agent labels in the Jenkinsfile.

Q: How many pipelines do you have in your project?

We don't use a single pipeline for everything. Each application or service has its own CI/CD pipeline, and infrastructure has a separate Terraform pipeline. For example, we had separate pipelines for frontend, backend, infrastructure, database migrations, and monitoring. Jenkins organized them through folders. The Jenkins controller orchestrated the jobs, while dedicated agents executed Terraform, Docker builds, tests, and deployments.

Q: How do agents connect and communicate with the master?

Agents connect to the master via SSH on Linux (most common) or via JNLP/WebSocket for inbound connections. The master keeps a persistent connection and sends stage instructions to available agents. Logs stream back in real time to the Jenkins UI. Each agent registers with a label — our Terraform agent has label 'terraform', Docker agent has label 'docker' — and the Jenkinsfile routes each stage to the right one.

Q: Why not run everything on the Jenkins master?

Security — infra credentials like AWS keys are isolated to the Terraform agent only. Performance — heavy Docker builds and Terraform runs don't overload the master. Tool isolation — Terraform and Docker can have separate environments without conflicts. Scalability — multiple jobs can run in parallel on different agents simultaneously.

Q: How do you handle multiple environments?

We use Git branch strategy — develop branch deploys to dev (no approval), release branch to test (approval required), main branch to production (manual approval with required reviewers). In the Jenkinsfile, when { branch 'main' } blocks ensure a stage only runs on the correct branch. Kubernetes namespaces (dev/test/prod) or separate EKS clusters provide the isolation.

Q: What happens when an agent is busy?

Jenkins queues the pipeline run until a matching agent becomes free. If you have 1 Terraform agent and 3 pipelines all need Terraform stages simultaneously, 2 will wait in the queue. The solution is to add more agents with the 'terraform' label — the controller automatically load-balances across all free matching agents.

Quick Reference Cheat Sheet

Commands and concepts at a glance

Essential Jenkinsfile patterns

```
agent none // no default agent
agent { label 'terraform' } // route stage to infra agent
agent { label 'docker' } // route stage to app agent
when { branch 'main' } // only run on main branch
input 'Approve deploy to prod?' // pause for human approval
environment { AWS_REGION = 'ap-south-1' } // env vars
```

Essential kubectl commands used in pipelines

```
kubectl config use-context dev # switch to dev cluster/context
kubectl apply -f k8s/ # deploy all manifests
kubectl rollout status deployment/app # wait for rollout to complete
kubectl get pods -n dev # check pod status
kubectl logs -f deployment/backend # stream application logs
```

Essential Terraform commands in Agent-1

```
terraform init # initialise providers & backend
terraform fmt # format code
terraform validate # syntax + logic check
terraform plan -var-file=env/dev.tfvars # preview changes
terraform apply -auto-approve # create/update resources
terraform destroy -auto-approve # tear down (careful!)
```

Key concept summary

Term	Definition
Jenkins Master	Controller server — orchestrates, never executes heavy jobs
Jenkins Agent/Slave	Worker server — executes pipeline stages
Label	Tag on agent ('terraform','docker') matched by Jenkinsfile agent{label}
Webhook	GitHub HTTP call to Jenkins when code is pushed
Namespace	Kubernetes logical partition for dev/test/prod isolation
ECR	Amazon Elastic Container Registry — stores Docker images
EKS	Amazon Elastic Kubernetes Service — managed Kubernetes
Approval gate	input step in Jenkinsfile — pauses pipeline for human review